

REAL-LIFE AI PROGRESSION SYSTEM

A beginner-to-production roadmap for a serious 3rd-year major project

Project vision

Build an AI-powered personal guidance and progression system that understands a user's current situation, identifies what needs attention, recommends the next useful action, and learns from the user's progress. The system covers physical wellness, education, mental wellness, emotional wellbeing, and career/life direction.

Inspired by the idea of a game-like personal System, but designed as an original product. The anime is the inspiration for progression mechanics; it is not the product or branding.

Target: strong college major project + portfolio project + deployable prototype + interview-ready technical story

1. Executive Overview

This project should be built as a real software product, not as a collection of AI features. The core problem is: many students and young adults have goals and problems but do not know what to prioritize, how difficult the next step should be, or how to recover when they are overloaded.

The application creates a personal profile, tracks several life domains, detects bottlenecks, generates an appropriate quest or recovery action, explains the recommendation, records evidence of completion, and adapts future recommendations.

The central product idea is:

"Given this person's goal, current state, constraints, and recent progress, what is the most useful next action right now?"

Why this is a good major project

- It solves a real problem rather than being only a visual demo.
- It gives you full-stack work: frontend, backend, database, APIs, authentication, analytics, deployment.
- It gives you meaningful AI work: recommendations, natural-language explanations, personalization, and later machine learning.
- It can be evaluated with measurable outcomes such as quest completion, prediction quality, retention, and user satisfaction.
- It can become a useful portfolio project even if you later narrow the product for a specific audience such as college students.

2. What You Are Actually Building

Do not define the project as an "AI chatbot" or a "Solo Leveling clone." Define it as an adaptive personal guidance and progression platform.

Five life domains

Domain	Purpose	Example signals
Physical	Support safe, sustainable wellness habits.	Activity, recovery, sleep routine, fitness logs
Educational	Improve learning and academic/technical progress.	Study sessions, quiz scores, coding problems, exam dates
Mental wellness	Support focus, workload management, rest and healthy coping.	Self-reported stress, energy, focus, breaks
Emotional wellbeing	Support self-awareness, reflection and connection.	Mood check-ins, journaling, social connection
Career / life direction	Convert long-term goals into roadmaps and milestones.	Target role, skill gaps, projects, deadlines

Important boundary

The app should support wellbeing, education and planning. It should not diagnose mental illness, medical conditions, or promise treatment. When a user reports serious or persistent distress, the system should move away from gamification and encourage appropriate professional or emergency support according to the situation.

3. Main User Journey

Your first version should create one complete loop before adding advanced features.

1. Create account
2. Complete onboarding
3. Complete baseline assessment
4. Set one or more goals
5. System creates initial profile
6. User checks in: energy / stress / mood / availability
7. System detects priority
8. System generates today's quest OR recovery plan
9. User records evidence/result
10. XP and progress update
11. System learns from the result
12. Next day starts with updated context

Example

User: "I have an exam in 20 days, I am weak in DSA, I feel stressed and I only have 90 minutes today."

System: identifies DSA as the highest-impact academic gap but reduces workload because stress and available time are low; it gives a short DSA task, a brief recovery action, and explains why.

4. MVP - The First Version You Must Finish

MVP means the smallest version that demonstrates the real product idea. Do not start with ML, social networking, voice assistants, or dozens of integrations.

Module	MVP contents	Must work?
Authentication	Register, login, logout, protected pages	Yes
Onboarding	Age range, role/student status, goals, availability, preferences	Yes
Assessment	Questions for five domains + goal setting	Yes
Status dashboard	Level, XP, domain scores, current priorities	Yes
Check-in	Energy, stress, mood, focus, time available	Yes
Priority engine	Chooses the most useful domain/action	Yes
Quest engine	Generates small actionable tasks	Yes
Completion evidence	Mark complete + optional result/score	Yes
Progress engine	XP, levels, streaks, historical trends	Yes
Explanation	Why this quest was selected	Yes
Safety layer	Non-diagnostic language + support escalation	Yes

Do not build first

- Mobile app + web app simultaneously
- Social feed
- Complex 3D game
- Wearable integrations
- Custom large language model
- Full autonomous agents
- Marketplace

- Advanced recommendation ML before you have user data

5. Baseline Assessment Design

The assessment is the first place where your project can become different from a simple habit app. It should produce a starting profile, not a medical diagnosis.

Area	Beginner questions	Possible derived score
Physical	Activity frequency, basic fitness ability, recovery habits, sleep routine, daily movement	Physical wellness score
Education	What are you learning? current confidence? recent quiz/test result? study consistency?	Learning score
Mental wellness	Focus, workload, perceived stress, recovery habits, distraction level	Focus/recovery score
Emotional	Current mood, emotional energy, reflection habits, social connection	Emotional wellbeing score
Career / life	Long-term goal, target role, deadline, current skills, biggest uncertainty	Goal readiness score

Scoring principle

Start with transparent, explainable weighted scores. Example: if an educational score uses consistency, demonstrated performance and goal relevance, the user should be able to understand where the number came from. Avoid pretending that an arbitrary score is scientifically validated.

Example educational score

```
learning_score = 0.35 * knowledge_evidence
+ 0.30 * consistency
+ 0.20 * goal_alignment
+ 0.15 * recent_improvement
```

Keep every component on a 0-100 scale.

6. Database Design - Beginner Version

Use a relational database. PostgreSQL is a strong choice for the final system; SQLite is fine while learning locally.

Table	Main fields	Purpose
users	id, email, password_hash, created_at	Authentication/account
profiles	user_id, age_band, role, preferences	Basic profile
domains	id, name	Five life domains
assessments	id, user_id, created_at, version	Assessment session
assessment_answers	assessment_id, question_id, value	Assessment responses
goals	id, user_id, title, domain, target_date, status	Long-term goals
skills	id, user_id, domain, name, level	Skills/competencies
checkins	id, user_id, energy, stress, mood, focus, time_available	Daily context

Table	Main fields	Purpose
quests	id, user_id, title, domain, difficulty, xp, rationale	Generated actions
quest_results	quest_id, completed, evidence, score, completed_at	Outcome/evidence
xp_events	id, user_id, reason, xp, created_at	Audit trail for XP
progress_snapshots	id, user_id, date, domain_scores, overall_score	Analytics history
support_resources	id, category, title, url, tags, safety_level	Curated recovery/help resources

7. System Architecture

Use a clean separation between the user interface, business logic, data, and AI services.

```
FRONTEND
React / HTML-CSS-JS
|
REST API
|
FASTAPI BACKEND
+-----+-----+
|||
Auth Service Progress Engine Recommendation Engine
|||
+-----+-----+
|
POSTGRESQL
|
Analytics / History Data
|
AI / LLM SERVICE
|
Explanations / Quest wording
```

Recommended stack

Layer	Recommended beginner-friendly choice	What you learn
Frontend	HTML/CSS/JavaScript first; React after MVP	UI, state, API calls
Backend	Python + FastAPI	REST APIs, validation, authentication
Database	PostgreSQL; SQLite during early learning	SQL, schema design, relationships
AI	LLM API with strict structured prompts	Prompting, structured outputs, guardrails
Analytics	Python, Pandas, Matplotlib	Data cleaning, trends, evaluation
ML later	Scikit-learn	Classification/regression/recommendation experiments
Deployment	Any mainstream cloud platform you can learn and document	DevOps basics, environment variables, monitoring
Version control	Git + GitHub	Branches, pull requests, release history

8. Scoring and Progression Engine

This is the first important algorithmic component. Build it yourself instead of asking an LLM to decide everything.

Domain score

Each domain has an explainable 0-100 score. Normalize components, apply weights, clamp to a valid range, and store the components so you can show a breakdown.

Overall score

```
overall = 0.22*physical
+ 0.22*education
+ 0.20*mental_wellness
+ 0.18*emotional_wellbeing
+ 0.18*career_direction
```

These weights are product choices, not medical or scientific truth. Let the user understand that the score is a gamified progress indicator.

XP system

XP example

```
small_quest = 20 XP
medium_quest = 40 XP
major_milestone = 100 XP
validated_learning_result = bonus XP
recovery_action = small XP or no XP
```

level_threshold(n) can grow as the level rises.

Never make the user feel that rest is failure. A recovery day can reduce workload without reducing their overall standing.

9. The Core USP - Priority / Bottleneck Engine

This is the component that can make your project technically different. Instead of asking "what tasks exist?", score possible actions against the current situation.

```
Priority(action) =
GoalImpact * 0.30
+ BottleneckImpact * 0.25
+ CurrentNeed * 0.20
+ Feasibility * 0.15
+ UserPreference * 0.10
```

Examples of input signals: deadline proximity, current skill gap, recent performance, stress/energy check-in, time available, recent workload, and previous quest success rate.

Bottleneck logic

Example:

```
Poor study performance
-> check sleep/recovery
-> check available time
-> check prerequisites
-> check recent workload
-> identify likely bottleneck
-> choose action that attacks the bottleneck
```

This is better than simply choosing the lowest score. A low score does not always mean "work harder on this area."

10. Adaptive Quest Engine

Quests should be specific, measurable, achievable, safe, and connected to a goal.

Quest input	Example
Goal	Pass DBMS exam in 20 days

Quest input	Example
Current gap	SQL joins weak
Available time	45 minutes
Current energy	5/10
Stress	6/10
Recent history	2 missed study quests this week
System decision	Short targeted study + brief recovery
Quest	25 minutes SQL joins + 10 practice questions
Evidence	Quiz score + mistakes + completion time

Difficulty adaptation

```

if success_rate >= 0.85 and recovery_ok:
    difficulty += 1 step
elif success_rate < 0.50 or repeated_overload:
    difficulty -= 1 step
else:
    difficulty stays the same

```

Use guardrails: minimum/maximum task size, domain-specific templates, prohibited unsafe advice, and fallback quests when the AI output fails validation.

11. Recovery & Support Mode

This feature comes directly from the original personal motivation for the project: sometimes a user does not need another challenge; they need help deciding how to recover and what to do next.

```

NORMAL MODE
-> check-in
-> if energy/stress indicators suggest overload
-> RECOVERY MODE

```

Recovery suggestions can include:

- short walk or gentle movement
- breathing / relaxation activity
- rest and hydration
- screen break
- journaling or reflection
- talk to a trusted person
- calming music/video/movie
- reduce today's workload

Then ask: "Did this help?"
Use the answer to adjust tomorrow's plan.

Content recommendation

For movies, videos, music, or articles, start with a small manually reviewed resource library tagged by mood/context, duration, intensity, and content type. Later you can build a recommendation model from user preferences. This is safer and easier to evaluate than letting an LLM invent links.

Escalation rule

When a user reports severe or urgent distress, suicidal thoughts, inability to stay safe, or symptoms that require professional attention, the system should prioritize immediate human/professional

support rather than gamified quests. Keep the exact emergency guidance country-aware and maintain a reviewed resource list.

12. Where AI/LLM Should and Should Not Be Used

Use the LLM for	Do not let the LLM be the only authority for
Natural-language explanations	Medical diagnosis
Rewriting a validated quest into friendly language	Mental-health diagnosis or treatment decisions
Generating multiple phrasing options	Arbitrary level/score calculations
Summarizing progress history	Raw database mutations without validation
Answering educational questions in a controlled context	Safety-critical decisions
Conversational check-ins	Unlimited autonomous behavior

Structured output

LLM input:
{user_context, allowed_domains, priority, constraints}

Expected JSON:
{
 "quest_title": "...",
 "steps": ["..."],
 "why": "...",
 "estimated_minutes": 25,
 "difficulty": "easy"
}

Backend validates every field before saving or showing it.

This design gives you a strong interview answer: the LLM is a component inside a governed system, not the system itself.

13. Machine Learning - Add It After You Have Data

Do not force ML into the project on day one. First build an interpretable baseline. Once you have enough anonymized interaction data or a carefully designed test dataset, you can compare ML against the baseline.

Stage	Technique	Purpose
Baseline	Weighted rules	Initial priority engine
Experiment 1	Classification	Predict whether a quest is likely to be completed
Experiment 2	Regression	Predict suitable task duration or difficulty
Experiment 3	Recommendation	Rank candidate next actions
Experiment 4	Personalization	Adjust recommendation weights per user cluster

Evaluation metrics

- Quest completion rate
- User-rated helpfulness
- Recommendation acceptance rate
- Prediction accuracy / precision / recall where applicable
- Reduction in repeated failed quests
- Retention over 7/30 days
- Goal milestone completion

14. Development Roadmap - 8 to 9 Months

Phase	Weeks	What you learn/build	Deliverable
Phase 0: Setup	1-2	Git, GitHub, VS Code, Python environment, basic project structure	Running backend + repository
Phase 1: Python + SQL foundation	3-5	Python functions/classes, APIs, SQL basics, CRUD	Small practice project
Phase 2: Backend	6-9	FastAPI, validation, auth, PostgreSQL	User/profile/goal APIs
Phase 3: Frontend	10-13	HTML/CSS/JS then React basics	Login + dashboard UI
Phase 4: Core system	14-18	Assessment, scoring, XP, levels, check-ins, priority rules	End-to-end MVP
Phase 5: AI	19-22	LLM API, prompt design, structured output, guardrails	AI quest + explanation engine
Phase 6: Adaptive system	23-27	Difficulty adaptation, bottlenecks, balance, recovery mode	Personalization v2
Phase 7: Analytics/ML	28-31	Pandas, evaluation, baseline vs model	Evaluation report
Phase 8: Deployment	32-35	Cloud deployment, environment variables, logging	Live demo
Phase 9: Polish	36	Testing, documentation, screenshots, presentation	Final major-project package

15. Your Weekly Learning Routine

Because you are learning from beginner level, consistency matters more than trying to master everything before coding.

Day	Main work	Typical time
Mon	Learn backend/Python concept	60-90 min
Tue	Implement one small feature	60-90 min
Wed	SQL/database practice	60 min
Thu	Frontend practice	60-90 min
Fri	Integrate + debug	60-90 min
Sat	Project deep work	2-4 hr
Sun	Review, document, plan next week	60-90 min

Rule: every week must end with something demonstrable. A screen, API, database query, test, algorithm, chart, or working feature. Avoid spending many weeks only watching tutorials.

16. Beginner Learning Order

Order	Topic	What you need to know before moving on
1	Python basics	Variables, conditions, loops, functions, lists/dicts, files, exceptions
2	OOP basics	Classes, objects, methods, inheritance basics
3	Git/GitHub	clone, add, commit, push, branch, merge, README
4	SQL	SELECT, INSERT, UPDATE, DELETE, JOIN, GROUP BY, indexes basics
5	HTTP/REST	request, response, JSON, GET/POST/PUT/DELETE, status codes
6	FastAPI	routes, schemas, dependency injection basics, authentication
7	Frontend	HTML, CSS, JavaScript, forms, fetch, local state
8	React	components, props, state, effects, API integration
9	AI integration	structured prompts, JSON output, validation, retry/fallback
10	Data/ML	Pandas, feature engineering, train/test split, evaluation
11	Deployment	environment variables, logs, database deployment, basic security

Do not wait until you "finish Python" to start the project. Learn each topic just before you need it and apply it immediately.

17. Suggested Project Structure

```
life-system/  
├── frontend/  
│   ├── src/  
│   ├── components/  
│   ├── pages/  
│   └── services/  
├── backend/  
│   ├── app/  
│   │   ├── api/  
│   │   ├── models/  
│   │   ├── schemas/  
│   │   ├── services/  
│   │   ├── core/  
│   │   └── main.py  
│   └── tests/  
├── data/  
├── docs/  
├── scripts/  
├── .env.example  
├── README.md  
└── docker-compose.yml # later
```

18. Testing, Security, and Safety

- Unit tests: scoring, XP calculations, level thresholds, priority ranking.
- API tests: authentication, validation, permissions, error responses.
- AI tests: schema validation, unsafe output detection, missing-field fallback, prompt regression tests.
- Database safety: parameterized queries/ORM, backups, migration history.
- Authentication: hashed passwords, secure session/JWT handling, no secrets in GitHub.

- Privacy: collect only what you need, let users delete data, avoid storing raw sensitive reflections unless necessary.
- Safety: clear wellness boundaries, support escalation, reviewed content recommendations, no shame-based penalties.

19. Main Screens

Screen	Purpose
1. Welcome	Explain the product and safety boundaries
2. Sign up / login	Account creation
3. Onboarding	Goals, preferences, available time, interests
4. Initial assessment	Questions across five domains
5. System status	Level, XP, five domains, balance, current focus
6. Daily check-in	Energy, stress, mood, focus, available time
7. Today's System	Main quest, side quest, recovery option, rationale
8. Quest detail	Steps, timer/checklist, evidence input
9. Progress	Trends, level history, skills, completed milestones
10. Goal roadmap	Long-term goal -> skills -> milestones -> quests
11. Recovery mode	Gentle support and reviewed resources
12. Settings/privacy	Account, data controls, preferences

20. How This Helps Your Job Profile

Your project becomes useful for interviews only when you can explain the engineering decisions and show evidence that the system works.

What to show	Evidence
GitHub	Clean commits, README, architecture diagram, API docs
Live demo	Working sign-up, assessment, dashboard, quest flow
Technical depth	Database schema, backend architecture, scoring engine
AI depth	Prompt templates, structured outputs, validation, evaluation
Data/ML depth	Baseline vs model comparison, charts and metrics
Product depth	Problem statement, user journey, personas, feedback
Safety	Wellness boundary design and escalation logic
Testing	Unit/API/AI test results
Presentation	2-minute demo + 5-minute technical explanation

21. Questions You Should Be Ready to Answer

- What real problem does this solve?
- Why is this not just a habit tracker?

- Why did you use an LLM?
- Which decisions are rule-based and which are AI-based?
- How does the priority engine select a quest?
- How do you prevent the AI from giving unsafe advice?
- How does the system adapt difficulty?
- How do you calculate XP and level?
- How do you measure whether recommendations are useful?
- Why PostgreSQL?
- How is authentication handled?
- How would you scale this to 100,000 users?
- How would you improve the system with machine learning?
- What are the limitations of your current version?
- What would you change after user testing?

22. Milestone Checklist

Milestone	Definition of done
M1	Python/Git/SQL basics demonstrated through mini tasks
M2	Backend can create users and goals
M3	Database schema works with migrations/seeding
M4	Frontend can call backend and show profile
M5	Assessment produces domain scores
M6	Dashboard shows status + XP + history
M7	Priority engine chooses a next action
M8	Quest engine produces validated quests
M9	User completion updates progress
M10	Check-in can switch the system to recovery mode
M11	AI explains recommendations
M12	Testing suite passes
M13	Deployed online
M14	User test with a small group + feedback report
M15	Final report + demo + GitHub README + architecture diagram

23. Final Recommendation for You

Start narrow: college students / young adults who are struggling with direction, consistency, workload and personal improvement. This is easier to understand, test, and demonstrate than trying to serve everyone.

Build the core loop first:

Assess -> Understand -> Prioritize -> Recommend -> Act -> Measure -> Adapt

Then add sophistication in this order:

- Bottleneck detection
- Adaptive difficulty
- Goal roadmaps
- Recovery mode
- Explainable recommendations
- Evidence-based educational progression
- Content recommendations
- ML personalization

Avoid copying names, characters, artwork, or exact anime interface elements. Use an original product identity and treat the game-like progression mechanic as inspiration.

Most importantly: do not let the project become a collection of features. The quality of the decision engine is the project.

One-sentence project pitch

"An adaptive AI personal guidance system that helps users improve across learning, physical wellness, mental wellness, emotional wellbeing, and career goals by continuously deciding the most useful next action and adjusting it from real user feedback."

24. Your First 30 Days - Exact Starting Plan

Days	Work	Output
1-3	Install VS Code, Python, Git; learn terminal basics; create GitHub repo	Working environment + first commit
4-7	Python basics: variables, conditions, loops, functions	5 small Python programs
8-10	Python data structures + files + exceptions	Mini CLI habit tracker
11-13	OOP + modules + virtual environment	Small Python app structure
14-16	SQL basics + SQLite/PostgreSQL setup	10 SQL practice queries
17-19	HTTP/REST concepts + FastAPI hello world	First API endpoints
20-22	Build users/goals CRUD API	Working backend mini-project
23-25	Start frontend basics and API connection	Simple dashboard
26-28	Design assessment questions + scoring formulas on paper	Assessment specification
29	Design database schema	ER diagram
30	Demo day: show what works + write what failed	Month-1 progress report

Remember

Start with one complete working loop: assess the user, choose a useful next action, record the result, and adapt the next recommendation. Every later feature should strengthen that loop.